

UNIVERSITY OF KARACHI
Department of Computer Science
Software Project Management — 506 (LAB 3+0)

CONGINE

Deterministic Output Governance for AI-Assisted Software Development

TECHNICAL SPECIFICATION

Software Requirements Specification and Architectural Design Document

Field	Detail
Document Reference	CGN-DOC-02
Project Name	Congine — Deterministic Output Governance Engine
Course	Software Project Management — 506 (LAB 3+0)
Instructor	Dr. Humera
Semester	6th
Assignment	Assignment-01: Project Charter Development
Team Members	1. _____ Roll No: _____ 2. _____ Roll No: _____ 3. _____ Roll No: _____ 4. _____ Roll No: _____ 5. _____ Roll No: _____
Version	1.0
Date of Submission	_____
Status	Submitted for Evaluation

Table of Contents

1. Introduction
2. Overall System Description
3. System Architecture
4. Component Specification
5. Functional Requirements
6. Non-Functional Requirements
7. Contract and Data Model
8. Integration Architecture
9. Technology Stack
10. Security Design
11. Quality Assurance and Testing Strategy
12. Deployment and Operational Considerations
13. Constraints, Assumptions and Dependencies
14. Requirements Traceability Matrix
15. Planned Technical Direction

1. Introduction

1.1 Purpose

This document specifies the software requirements and architectural design of Congine, a deterministic output governance engine for AI-assisted software development. It is intended to serve as the authoritative technical reference for the development team, and as the basis against which the delivered system is verified.

The document combines the content conventionally distributed between a Software Requirements Specification and an Architectural Design Document. This consolidation is deliberate: the system's non-functional requirements and its architectural structure are unusually tightly coupled, and separating them would obscure the reasoning that connects them.

1.2 Scope

The specification covers the core validation engine and its immediately supporting infrastructure. It defines the system's architecture, its functional and non-functional requirements, its data model, its integration surfaces and its security design.

The following are explicitly outside the scope of this document: the centralized management service through which contracts may be distributed in a multi-team deployment; graphical user interfaces; commercial packaging; and detailed algorithmic implementation of individual components. Where a future capability is referenced for context, it is identified as such.

1.3 Intended Audience

Audience	Relevant Sections	Purpose of Reading
Development team	All sections	Implementation reference and verification basis
Project supervisor / evaluator	1, 2, 3, 5, 6	Assessment of technical soundness and completeness
Integrating developer	2, 7, 8	Understanding how to adopt the system in an application
Security reviewer	6.3, 10	Assessment of the threat model and defensive measures
Quality assurance	5, 6, 11, 14	Derivation of test cases and traceability of requirements

1.4 Definitions and Acronyms

Term	Definition
Adapter	A concrete implementation of a port, providing a specific technology-bound realization of an abstract capability.
Breach	A single identified instance of an output failing to satisfy a specific requirement expressed in a contract.
Composition root	The single location in the system at which concrete implementations are constructed and assembled.
Contract	A machine-readable specification of the requirements an output must satisfy.
Enforcement policy	The configured response applied when validation identifies one or more breaches.
Hot path	The sequence of operations executed on every validation request; the performance-critical path.

Term	Definition
Load shedding	The controlled rejection of work when the system is operating at capacity, in preference to unbounded queueing.
Port	An abstract interface defining a capability the system requires, without specifying its implementation.
SDK	Software Development Kit — the distributable form in which the system is consumed by a host application.
Validation result	The structured outcome of evaluating an output against a contract.

1.5 References

16. CGN-DOC-01 — Congine Project Concept Brief, Version 1.0
17. CGN-DOC-03 — Congine Business Case and Project Management Plan, Version 1.0
18. IEEE Std 830 — Recommended Practice for Software Requirements Specifications
19. ISO/IEC 25010 — Systems and Software Quality Requirements and Evaluation
20. Cockburn, A. — Hexagonal Architecture (Ports and Adapters)
21. Nygard, M. — Release It! Design and Deploy Production-Ready Software
22. OWASP Top 10 for Large Language Model Applications

2. Overall System Description

2.1 Product Perspective

Congine is delivered as a software development kit that is embedded within a host application or invoked as a service by an AI coding assistant. It is not a standalone application and does not provide a user interface. It is a library and service component whose value is realized through integration.

This positioning imposes a defining constraint on the design. The system operates within a process it does not own, and it must therefore never destabilize that process. A validation engine that introduces unpredictable latency, exhausts resources, or propagates unexpected failures into its host would not be adopted regardless of the quality of its validation logic. A substantial proportion of the architecture is directed at satisfying this constraint.

2.2 Principal Functions

Function	Description
Contract acquisition	Load contract definitions from a local directory or a remote source, and retain them in memory for rapid access.
Output validation	Evaluate a supplied output against the applicable contract using a fixed sequence of deterministic rules.
Verdict production	Produce a structured result identifying conformance and, where applicable, each individual breach.
Enforcement	Apply the configured enforcement policy: permit, permit with warning, or block.
Observation	Emit a record of every validation performed, without blocking the validation path.
Resilience management	Continue correct operation when external dependencies are slow or unavailable.
Isolation	Maintain separation of state and configuration where multiple tenants share a single process.

2.3 User Classes

- **Integrating developer.** Embeds the SDK within an application and configures its behaviour. Requires clear interfaces, predictable failure semantics and comprehensible documentation.
- **Contract author.** Writes and maintains the contracts against which outputs are evaluated. Requires an expressive but unambiguous contract format and clear feedback where a contract is malformed.
- **Operator.** Runs the host application in production. Requires observability, controlled failure behaviour and configuration appropriate to the deployment environment.
- **AI coding assistant.** A non-human consumer that requests validation during its own operation. Requires machine-readable results structured for direct consumption rather than human presentation.

2.4 Operating Environment

Aspect	Requirement
Language runtime	A current supported release of the Python runtime.
Operating system	Platform-independent; primary targets are Linux and macOS, with Windows support intended.

Aspect	Requirement
Deployment form	Embedded library within a host process; optionally exposed as a local service to an AI assistant.
Network dependency	Optional. The system is required to operate fully in an offline configuration.
Persistent storage	Local filesystem access for cached contract definitions.
Concurrency	Must operate correctly under concurrent access from multiple threads and from asynchronous callers.

3. System Architecture

3.1 Architectural Style

The system adopts a layered hexagonal architecture, also described as the ports and adapters pattern. The selection of this style is a direct consequence of two requirements: the validation logic must remain independent of any specific external technology, and new capabilities must be addable without modification to the components that have already been verified.

The organizing principle of the style is the separation of policy from mechanism. The rules governing what constitutes a conforming output are policy, and reside at the centre of the system. The means by which contracts are retrieved, results are recorded and work is scheduled are mechanism, and reside at the periphery. The centre defines the capabilities it requires as abstract interfaces; the periphery provides concrete implementations of those interfaces.

The Governing Architectural Rule

Dependencies point inward only. An outer layer may depend upon an inner layer; an inner layer may never depend upon an outer layer. This single rule is what permits new external capabilities to be introduced without alteration of the verified core, and it is enforced by automated checks rather than by convention alone.

3.2 Layer Model

The system is organized into six layers, numbered from the innermost outward.

Layer	Designation	Responsibility	Permitted Dependencies
L0	Kernel	Configuration structures, error type hierarchy, security limits, data sanitization utilities	None
L1	Ports	Abstract interface definitions expressing capabilities the core requires	L0 only
L2	Domain	Validation rules and immutable value objects; contains no input or output operations of any kind	L0, L1
L3	Use Cases	Orchestration of complete operations; coordinates collaborators without performing the work itself	L0, L1, L2
L4	Infrastructure	Concrete implementations of ports: caching, scheduling, retrieval, recording, resilience	L0, L1
L5	Adapters	Entry points and the composition root; the only layer that constructs concrete implementations	All

3.2.1 Rationale for the Layer Boundaries

The placement of the validation rules in L2, isolated from all input and output, is the most consequential structural decision in the system. It produces three properties that follow directly. The rules are trivially testable, requiring no test infrastructure beyond ordinary function invocation. The rules are inherently deterministic, since they cannot depend on network conditions, filesystem state or elapsed time. And the rules are stable, since the technologies most likely to change over the life of the system all reside in L4 and are reachable only through the abstract interfaces defined in L1.

The designation of a single composition root in L5 is similarly deliberate. Because it is the only location in which concrete implementations are constructed, the complete assembled structure of the system can be understood by reading one module, and the substitution of any implementation — for testing or for a change of technology — requires modification in one place.

3.3 Structural Diagram

	Layer Composition (outermost to innermost)
L5	Entry points Composition root Framework integrations
↓	depends upon
L4	Schema cache Bounded executor Resilience controller Contract sources Event recorder Structured logger
↓	depends upon (through L1 interfaces only)
L3	Validation orchestration Contract synchronization
↓	depends upon
L2	Rule engine Validator strategies Immutable value objects
↓	depends upon
L1	Abstract capability interfaces (ports)
↓	depends upon
L0	Configuration Error hierarchy Security limits Sanitization

3.4 The Ports and Adapters Mechanism

A port is a declaration of a required capability, expressed as an abstract interface without implementation. An adapter is a concrete realization of that capability. The core of the system depends exclusively upon ports and is unaware of which adapter has been supplied.

The practical consequence is substitutability. A cache implemented in process memory and a cache implemented against a distributed store satisfy the same port; the orchestration layer that consumes the port is unaffected by the substitution. The same property is exploited in testing, where lightweight test implementations replace production adapters without any modification to the code under test.

Port	Capability Declared	Obligations of Any Implementation
Contract storage	Retrieve and store contract definitions by identifier	Thread-safe; expired entries must be reported as absent
Contract source	Obtain the set of active contracts; persist and restore a local copy	Must not fail the calling operation when the source is unavailable
Execution runner	Execute a validation function within a bounded time allowance	Enforce capacity limits and time bounds; support re-entrant invocation
Event recorder	Record a validation event for later analysis	Must not block the caller; may discard events under sustained pressure
Resilience controller	Track the health of an external dependency and gate access to it	Implement a closed / open / trial state progression

Port	Capability Declared	Obligations of Any Implementation
Extended validator	Perform validation beyond the fixed rule set	Return a flat collection of identified breaches
Logger	Emit structured diagnostic records	Redact sensitive fields unconditionally

4. Component Specification

This section describes the principal components of the system, their responsibilities and the requirements they satisfy. Implementation detail is deliberately confined to the level necessary to establish the architecture.

4.1 Rule Engine (L2)

The rule engine is the component that performs the actual evaluation of an output against a contract. It comprises a fixed sequence of independent rules, each of which examines one category of requirement and reports any breaches identified.

The rules address, in order: the presence of required fields; the conformance of field values to their declared types; the membership of constrained values within their permitted sets; the containment of numeric values within permitted ranges; the prohibition of absent values where a value is mandatory; and the conformance of textual values to declared structural patterns.

The engine performs no input or output of any kind. It accepts two data structures and returns a collection of identified breaches. This purity is the foundation of the system's determinism and is treated as an invariant that may not be violated by any subsequent development.

Rule Ordering

The rules are evaluated in a defined sequence because certain checks are meaningless unless earlier checks have succeeded. The type of a field that is absent cannot be evaluated, and the range of a value that is not numeric cannot be evaluated. Each rule therefore defers to its predecessors and reports only breaches within its own area of responsibility, ensuring that a single underlying fault produces one clear breach rather than several confusing ones.

4.2 Validation Orchestration (L3)

The orchestration component coordinates a complete validation operation. It does not perform validation itself; it sequences the participants. Its responsibilities are: verification that the supplied output falls within permitted size limits; retrieval of the applicable contract; delegation of the evaluation to the execution runner under a bounded time allowance; recording of the outcome; and application of the configured enforcement policy.

One ordering property within this sequence is treated as a guarantee rather than an implementation detail. The record of an evaluation is emitted before the enforcement policy is applied. The consequence is that a violation remains observable even in the case where enforcement blocks execution and the resulting error is subsequently caught and discarded by the host application. An audit record that a violation occurred is considered more valuable than an error indicating that something failed.

4.3 Contract Cache (L4)

Contract definitions are retained in an in-process cache so that validation does not require retrieval from an external source. The cache applies a frequency-based eviction policy with time-based expiry, retaining frequently consulted contracts and discarding those that are rarely used.

The cache is required to support concurrent access from the validation path and from the background process that refreshes contract definitions. A requirement of particular importance is that refreshing the cache must update entries in place rather than clearing and repopulating it. A clear-then-repopulate strategy would create an interval during which a concurrent validation observes an empty cache and fails, despite the contract being present both before and after the refresh.

4.4 Bounded Execution Runner (L4)

The execution runner is the component responsible for the system's latency guarantee. It maintains a fixed pool of execution capacity together with a bounded permit allocation. A validation request that cannot obtain a permit is rejected immediately rather than queued, and a request that exceeds its time allowance returns a degraded result to the caller.

Two design properties warrant explicit statement. First, the rejection of excess work is preferred to unbounded queueing, because an unbounded queue converts a capacity problem into a latency problem and ultimately into a memory-exhaustion problem. Second, capacity that has been allocated to work exceeding its time allowance is not reclaimed until that work genuinely terminates. Reclaiming it earlier would permit more concurrent work than the system was designed to sustain. The caller receives a timely degraded result while the system's accounting of its own capacity remains accurate.

4.5 Resilience Controller (L4)

Access to external dependencies is mediated by a resilience controller implementing a three-state progression. In the closed state, requests proceed normally and consecutive failures are counted. On reaching a configured threshold the controller enters the open state, in which requests are rejected immediately without an attempt being made. After a configured interval it enters a trial state, in which a single request is permitted; success returns the controller to the closed state and failure returns it to the open state.

The purpose is to prevent an unavailable dependency from consuming the time budget of every subsequent request. Without such a control, a dependency that has become unresponsive imposes its full timeout on every caller, converting a partial failure into a systemic one.

4.6 Contract Sources (L4)

Two implementations of the contract source port are provided. The first reads contract definitions from a local directory, requiring no network access and intended for development, testing and self-contained deployment. The second retrieves definitions from a remote service and maintains a local copy so that operation continues when the service is unreachable.

The local copy is written using an atomic replacement procedure so that an interrupted write cannot leave a partially written file in place, and it is subjected to integrity verification before use. Locally cached data is treated with the same suspicion as data received over a network.

4.7 Event Recorder (L4)

Validation events are recorded through a non-blocking mechanism. Events are placed on a bounded internal queue and transmitted by a background process; the recording call returns immediately without awaiting transmission. Where the queue is full, events are discarded and the discard is counted and reported.

The design accepts the loss of observability data in preference to the delay of validation. A configuration in which recording is disabled entirely is also supported, and in that configuration no background process and no network client are created.

4.8 Composition Root (L5)

The composition root constructs and assembles the complete object graph according to the supplied configuration. It selects the appropriate adapter for each port, provides each component with its dependencies, manages the lifecycle of background processes, and exposes an aggregated health status.

It is additionally responsible for maintaining separation between tenants where a single process serves several. Each tenant receives an independent set of infrastructure components so that the load, failure conditions and cached data of one tenant cannot affect another.

5. Functional Requirements

Requirements are identified as FR-n and are stated in a form permitting direct verification. Priority is expressed as Essential, Expected or Desirable.

5.1 Contract Management

ID	Requirement	Priority
FR-01	The system shall load contract definitions from a configured local directory.	Essential
FR-02	The system shall load contract definitions from a configured remote source.	Expected
FR-03	The system shall retain loaded contract definitions in memory for retrieval without external access.	Essential
FR-04	The system shall maintain a local copy of remotely obtained definitions and use it when the remote source is unavailable.	Expected
FR-05	The system shall refresh contract definitions periodically without interrupting concurrent validation.	Expected
FR-06	The system shall report the number of contract definitions successfully loaded during initialization.	Essential
FR-07	The system shall reject a contract definition exceeding the configured maximum size.	Essential

5.2 Validation

ID	Requirement	Priority
FR-08	The system shall evaluate a supplied output against a contract identified by reference.	Essential
FR-09	The system shall verify the presence of every field declared as required.	Essential
FR-10	The system shall verify that each present field conforms to its declared type.	Essential
FR-11	The system shall verify that each constrained field holds a value within its permitted set.	Essential
FR-12	The system shall verify that each numeric field lies within its declared range.	Essential
FR-13	The system shall verify that fields declared as mandatory do not hold an absent value.	Essential
FR-14	The system shall verify that each textual field conforms to its declared structural pattern.	Essential
FR-15	The system shall produce an identical verdict for identical combinations of output and contract.	Essential
FR-16	The system shall report every breach identified, not merely the first.	Essential
FR-17	The system shall reject an output exceeding the configured maximum size prior to evaluation.	Essential
FR-18	The system shall optionally perform extended validation beyond the fixed rule set when so configured.	Expected
FR-19	The system shall raise a distinct, identifiable error when the referenced contract is not available.	Essential

5.3 Enforcement

ID	Requirement	Priority
FR-20	The system shall support a permissive policy under which a non-conforming output proceeds with the outcome recorded.	Essential
FR-21	The system shall support a strict policy under which a non-conforming output is blocked by a raised error.	Essential
FR-22	The system shall support an observational policy under which validation is performed and recorded but never blocks.	Expected
FR-23	The system shall permit the enforcement policy to be configured without modification of application code.	Essential
FR-24	The system shall return a structured result identifying each breach in a form suitable for machine consumption.	Essential

5.4 Integration

ID	Requirement	Priority
FR-25	The system shall provide a declarative mechanism for applying validation to an existing function without altering its body.	Essential
FR-26	The system shall support both synchronous and asynchronous invocation with equivalent semantics.	Essential
FR-27	The system shall permit extraction of the value to be validated from a composite return value.	Expected
FR-28	The system shall be operable entirely offline with no network dependency.	Essential
FR-29	The system shall expose validation as a service consumable by an external AI coding assistant.	Expected
FR-30	The system shall provide a command-line interface suitable for use in automated pipelines.	Desirable

5.5 Observation and Operation

ID	Requirement	Priority
FR-31	The system shall emit a record of every validation performed, irrespective of outcome.	Essential
FR-32	The system shall emit the record before applying the enforcement policy.	Essential
FR-33	The system shall expose an aggregated health status describing its operational condition.	Expected
FR-34	The system shall emit structured diagnostic records at configurable severity levels.	Essential
FR-35	The system shall count and report events discarded under sustained load.	Expected
FR-36	The system shall permit recording to be disabled entirely, in which case no background process is created.	Essential
FR-37	The system shall maintain a durable record of validation events across process restarts.	Desirable

6. Non-Functional Requirements

Non-functional requirements are identified as NFR-n. For a system of this category they are of equal or greater consequence than the functional requirements, because they determine whether the system is adoptable at all.

6.1 Performance

ID	Requirement	Verification Method
NFR-01	A validation operation shall complete within a configurable time allowance, defaulting to a value in the low hundreds of milliseconds.	Timed measurement under representative load
NFR-02	Contract retrieval from the cache shall exhibit constant-time behaviour independent of the number of contracts held.	Comparative measurement across cache sizes
NFR-03	Exceeding the time allowance shall return a degraded result rather than extending the wait.	Fault injection with an artificially delayed rule
NFR-04	Concurrent validation requests exceeding capacity shall be rejected immediately rather than queued.	Saturation testing with measurement of rejection latency
NFR-05	Recording a validation event shall not block the validation path.	Measurement with an unavailable recording destination

6.2 Reliability and Availability

ID	Requirement	Verification Method
NFR-06	The system shall not propagate unexpected failures into the host application.	Fault injection at each external boundary
NFR-07	Initialization shall complete within a bounded interval irrespective of the availability of external services.	Timed initialization against an unreachable service
NFR-08	An unavailable external service shall not cause loss of previously loaded contract definitions.	Failure simulation with verification of cache contents
NFR-09	Simultaneous initialization of multiple instances shall not produce a coordinated surge of requests to an external service.	Multi-process initialization with request counting
NFR-10	Locally cached data shall be verified for integrity before use.	Corruption and substitution testing
NFR-11	The system shall release all resources deterministically on shutdown.	Resource inspection following termination

6.3 Security

ID	Requirement	Verification Method
NFR-12	Pattern evaluation shall employ an engine providing execution time linear in input length.	Adversarial pattern and input testing
NFR-13	All externally supplied input shall be bounded in size prior to processing.	Boundary testing at each documented limit
NFR-14	Potentially sensitive values shall be redacted before inclusion in any record or diagnostic output.	Inspection of emitted records containing sensitive input

ID	Requirement	Verification Method
NFR-15	Credentials shall never appear in diagnostic output.	Automated inspection of all emitted diagnostics
NFR-16	Where multiple tenants share a process, no tenant shall access the state of another.	Cross-tenant access testing
NFR-17	Locally cached data shall be rejected where its provenance or integrity cannot be established.	Substitution and permission-manipulation testing
NFR-18	The system shall refuse to operate against a non-local destination without appropriate credentials.	Configuration validation testing

6.4 Maintainability and Portability

ID	Requirement	Verification Method
NFR-19	Dependencies shall point inward only across all layer boundaries.	Automated dependency analysis
NFR-20	Concrete implementations shall be constructed only within the composition root.	Static inspection
NFR-21	The validation core shall perform no input or output operations.	Static inspection and dependency analysis
NFR-22	Every configuration option shall demonstrably affect the component it purports to control.	Configuration wiring test suite
NFR-23	Externally imposed dependencies shall be minimized; optional capabilities shall be separately installable.	Dependency manifest review
NFR-24	Components critical to the validation path shall be covered by both ordinary and adversarial tests.	Coverage measurement and test inventory

7. Contract and Data Model

7.1 The Contract

A contract is a structured document comprising an identifier and a specification of requirements. The specification declares the fields that must be present, the type each field must hold, the permitted values of constrained fields, the permitted ranges of numeric fields, and the structural patterns to which textual fields must conform.

Contracts are ordinary text documents suitable for storage in version control, review through the organization's established review process, and modification without alteration of application code. This is a deliberate property: the standard being enforced must be visible, reviewable and owned by the engineering team.

A Necessary Caution Regarding Contract Expressiveness

The rule engine implements a specific and finite set of requirement categories. A contract may contain declarations outside that set, and such declarations are not evaluated by the rule engine. This is a significant consideration for contract authors: a contract containing an unevaluated declaration will report conformance even where that declaration is not satisfied. The system therefore provides, and the project documents, both an explicit statement of the evaluated set and an optional extended validation capability that evaluates the wider specification. Ensuring that authors are aware of this distinction is treated as a safety requirement rather than a documentation preference.

7.2 Principal Data Structures

Structure	Purpose	Principal Attributes
Breach	Records a single identified requirement violation	Rule identity; affected field; descriptive message
Validation result	Records the outcome of one complete evaluation	Status; collection of breaches; degradation indicator; elapsed duration
Validation event	Records an evaluation for later analysis	Contract identity and version; status; duration; sanitized breach summary; timestamp
Configuration	Holds the complete runtime configuration	Approximately forty-five options governing every configurable aspect of behaviour

All result and event structures are immutable once constructed. Immutability eliminates an entire category of concurrency defect in which a result observed by one component is altered by another before it is acted upon, and it permits results to be shared freely across threads without synchronization.

7.3 Configuration Model

Configuration is represented as a single immutable structure constructed at initialization, conventionally from environment variables. Immutability guarantees that every component observes the same configuration for the lifetime of the process.

The system applies a completeness discipline to configuration: every declared option must demonstrably reach the component it governs. An option that is declared but not connected constitutes a defect, because it presents an operator with a control that does not function. This property is verified by an automated test rather than by review.

8. Integration Architecture

8.1 Two Integration Modes

The system supports two distinct integration modes, reflecting the two ways in which AI systems are used in practice. The distinction is architecturally significant because it determines which component holds control of the operating loop.

Property	Direct Integration	Tool-Based Integration
Applicable when	The application invokes an AI model directly from its own code	A self-contained AI assistant operates independently of the application
Position of the system	Positioned between the application and the model	Exposed as a service the assistant consults
Holder of control	The system, which may retry the interaction	The assistant, which requests a verdict and acts upon it
Mechanism	Declarative wrapping of the invoking function	A service interface offering validation operations
Principal advantage	Complete control of the corrective cycle	Compatible with widely used self-contained assistants
Principal limitation	Requires the application to invoke the model itself	The assistant is not compelled to consult the service

8.2 Direct Integration

In direct integration, validation is applied declaratively to an existing function. The function's body is unaltered; validation is applied to its return value. This satisfies the requirement that adoption should not necessitate restructuring of existing code.

Three response behaviours are supported: return of the original value together with the validation result; return of the original value alone, with enforcement governed by the configured policy; and raising of an error where the output does not conform. The selection of behaviour is a property of the integration point rather than of the global configuration, permitting different degrees of strictness at different points within a single application.

8.3 Tool-Based Integration

In tool-based integration, the system is exposed as a service offering validation operations that an AI assistant may invoke during its own operation. The assistant retains control of its process; the system supplies verdicts on request.

The architectural consequence of this arrangement is worth stating explicitly. The system cannot compel a self-contained assistant to consult it, because such assistants modify files directly rather than routing their output through an intermediary. Adoption therefore depends on the assistant being instructed to consult the service as part of its operating procedure. This is a property of the integration model rather than a deficiency of the design, and it is accommodated by supplying an instruction template alongside the service.

8.4 Supporting Enforcement Points

Because tool-based integration is advisory rather than compulsory, the architecture contemplates additional enforcement at points where compliance can be made mandatory: a pre-commit verification stage within the version control workflow, and a verification stage within the continuous integration pipeline. These constitute a defence in depth in which the earlier stages are convenient and the later stages are authoritative.

9. Technology Stack

Element	Selection	Rationale
Implementation language	Python	Predominant in AI-adjacent tooling; matches the intended integration environment
Pattern evaluation	A finite-automaton regular expression engine	Provides execution time linear in input length, eliminating a class of denial-of-service vulnerability by construction rather than by mitigation
Extended validation	A standard schema validation library	Avoids reimplementation of a well-specified standard; used only where extended validation is enabled
Transport	A modern HTTP client supporting synchronous and asynchronous operation	Required for both invocation styles without duplicated logic
Inter-process coordination	An advisory file locking library	Coordinates initialization across processes without requiring a shared service
Testing	A standard test framework with coverage measurement	Established tooling; supports the adversarial testing the design requires
Static analysis	A linter, a type checker and a security scanner	Automated enforcement of conventions and detection of common defect categories

A general principle governs these selections: the number of externally imposed dependencies is minimized. Capabilities not required by every consumer are made separately installable. This reduces the surface exposed to supply-chain risk and lowers the cost of adoption for a consumer who requires only the core function.

10. Security Design

10.1 Position

Congine is a governance component, and a governance component that is itself insecure is worse than none, because it establishes unwarranted confidence. Security is therefore addressed as a design property rather than as a subsequent hardening activity.

10.2 Trust Boundaries

Data crosses into the system at several points, and each is treated as untrusted.

Boundary	Data Received	Protective Measure
Output under validation	Arbitrary structured data produced by an AI system or application	Size bounded prior to processing; content never executed or interpreted
Contract definitions	Requirement specifications, including structural patterns	Size bounded; pattern length bounded; linear-time pattern evaluation
Remote source responses	Contract definitions retrieved over a network	Response size bounded; structure verified before acceptance
Locally cached data	Previously retrieved definitions read from local storage	Integrity and provenance verified before use
Configuration	Operational settings from the environment	Validated for coherence; incoherent configuration rejected at initialization

10.3 Principal Defensive Measures

- **Structural elimination of pattern-based denial of service.** Pattern evaluation employs an engine whose execution time is linear in the length of the input. The class of attack in which a crafted pattern and input combine to consume disproportionate processing time is eliminated by the choice of engine rather than mitigated by heuristics.
- **Comprehensive input bounding.** Every externally supplied input is subject to a documented size limit enforced before processing. Limits are centrally defined so that they may be adjusted coherently.
- **Capacity limitation with controlled rejection.** The bounded execution model prevents resource exhaustion under load, whether that load is accidental or deliberate.
- **Sanitization of recorded data.** Breach descriptions may contain fragments of the data being validated, and such data may be sensitive. Sanitization is applied before any record is emitted and is not optional.
- **Unconditional redaction of credentials.** Diagnostic output applies an unconditional redaction list to fields whose names indicate sensitive content, providing a safety net independent of developer diligence.
- **Verification of locally cached data.** Cached data is subject to integrity and provenance verification before use, on the principle that locally stored data merits the same suspicion as data received over a network.
- **Isolation between tenants.** Where multiple tenants share a process, each receives independent infrastructure and separately scoped storage, so that neither the load nor the data of one tenant is observable by another.

10.4 Threat Assessment Summary

Threat	Assessed Impact	Design Response
Pattern-based resource exhaustion	High — host application rendered unresponsive	Linear-time evaluation engine; pattern and input length bounds
Oversized input	Medium — disproportionate resource consumption	Size verification prior to any processing
Sensitive data disclosure through records	High — confidentiality breach	Mandatory sanitization; unconditional redaction of credential-bearing fields
Substitution of locally cached data	High — subversion of the enforced standard	Integrity and provenance verification before use
Cross-tenant data exposure	High — confidentiality breach between customers	Independent infrastructure and separately scoped storage per tenant
Dependency compromise	Medium — introduction of malicious code	Minimized dependency surface; version pinning; automated vulnerability scanning
Unavailability of external services	Medium — loss of function	Resilience controller; locally cached fallback; bounded initialization

11. Quality Assurance and Testing Strategy

11.1 Approach

The testing strategy reflects the system's position as a component embedded within applications it does not control. Failures in such a component are consequential, and the confidence required before deployment is correspondingly high.

Category	Objective	Scope
Unit testing	Verify each component in isolation	Every rule; every infrastructure component; every value structure
Integration testing	Verify components operate correctly in combination	Complete validation paths; initialization sequences; teardown
Adversarial testing	Verify correct behaviour under deliberately hostile conditions	Oversized inputs; malicious patterns; capacity saturation; corrupted cached data
Concurrency testing	Verify correctness under simultaneous access	Concurrent cache access; capacity accounting; state transitions
Failure injection	Verify graceful degradation	Unavailable services; corrupted data; exceeded time allowances
Configuration testing	Verify every option affects the component it governs	Complete configuration surface
Static analysis	Detect defects without execution	Linting; type verification; security scanning; dependency vulnerability review

11.2 Testing of Architectural Invariants

Certain architectural properties are verified by automated test rather than by review, on the principle that a property enforced only by convention will eventually be violated.

- Dependency direction is verified by automated analysis of the import structure.
- The purity of the validation core is verified by confirming the absence of input and output dependencies.
- Exclusive construction of concrete implementations within the composition root is verified by static inspection.
- Completeness of configuration wiring is verified by a dedicated test suite.

11.3 Substitutability in Testing

Because the core depends exclusively on abstract interfaces, testing employs lightweight substitutes in place of production adapters. These substitutes need not inherit from the interfaces they satisfy; structural conformance is sufficient. This property serves as a continuous demonstration that the abstractions are genuine rather than nominal: an interface that cannot be satisfied by a simple substitute is not a true abstraction.

11.4 Acceptance Criteria

Criterion	Threshold
Test coverage of the validation path	Comprehensive coverage of all components on the validation path

Criterion	Threshold
Adversarial coverage	Every documented limit exercised at its boundary
Static analysis	No unresolved findings; every accepted exception documented
Architectural invariants	All verified by automated test
Behavioural guarantees	Each supported by at least one dedicated test
Dependency vulnerabilities	No known unresolved vulnerabilities of high severity

12. Deployment and Operational Considerations

12.1 Deployment Configurations

Configuration	Characteristics	Intended Use
Self-contained	Contracts read from a local directory; recording disabled; no network access	Development; testing; air-gapped or restricted environments
Locally managed	Contracts read from a local directory; recording enabled	Small teams maintaining contracts alongside source code
Centrally managed	Contracts retrieved from a remote source with local fallback; recording enabled	Larger organizations requiring central governance of standards
Shared multi-tenant	Independent state per tenant within a single process	Service providers operating on behalf of multiple customers

12.2 Initialization

Initialization loads contract definitions and starts any configured background processes. The design requires initialization to complete within a bounded interval irrespective of the availability of external services; where a service is unreachable, locally cached definitions are used and initialization proceeds.

Where several instances initialize simultaneously, a coordination mechanism ensures that only one instance retrieves definitions from the external source while the remainder await its completion and use the result. Without such coordination, a coordinated restart would produce a simultaneous surge of requests against the external service at precisely the moment it is least able to absorb it.

12.3 Observability

The system exposes an aggregated health status describing the number of contract definitions held, current and rejected validation load, the state of the resilience controller, and the volume of recorded and discarded events. This is intended for consumption by the host application's existing monitoring arrangements rather than by a dedicated interface.

12.4 Failure Behaviour

The system's behaviour under failure is specified rather than incidental. External service unavailability results in use of locally cached definitions. Exceeding the time allowance results in a degraded verdict rather than an extended wait. Capacity exhaustion results in immediate rejection rather than queueing. Unavailability of the recording destination results in discarded events rather than a blocked validation path. In every case the host application continues to operate.

13. Constraints, Assumptions and Dependencies

13.1 Constraints

ID	Constraint	Consequence for the Design
C-01	The system operates within a process it does not own	Resource consumption must be bounded and failures must not propagate
C-02	Validation occurs on the development or request path	A strict latency budget applies to every operation
C-03	Determinism is required	No probabilistic component may participate in the validation decision
C-04	Self-contained AI assistants cannot be intercepted	A consultative service model is required in addition to direct integration
C-05	Offline operation must be supported	Every external dependency requires a local fallback
C-06	The dependency surface must remain small	Optional capabilities must be separately installable
C-07	The project is delivered by a small team with limited availability	Scope must be sequenced strictly and prioritized by dependency

13.2 Assumptions

ID	Assumption	Consequence if Invalid
A-01	Organizations possess standards that can be expressed as machine-readable rules	The contract mechanism would require substantial extension
A-02	Engineering teams will author and maintain contracts	Adoption would depend on automated contract inference
A-03	AI assistants will consult an available advisory service when instructed	Reliance would shift entirely to mandatory enforcement points
A-04	The applicable rule categories cover the majority of practical requirements	Extended validation would become the default rather than an option
A-05	A latency allowance in the low hundreds of milliseconds is acceptable	The performance target would require revision

13.3 Dependencies

- **External libraries.** A small set of third-party libraries providing pattern evaluation, transport, schema validation and inter-process coordination. Versions are pinned and subject to automated vulnerability review.
- **Runtime environment.** A current supported release of the Python runtime.
- **Optional external services.** Where central management is configured, a service supplying contract definitions. The system is required to operate correctly in its absence.
- **Integration protocol.** Tool-based integration depends upon the interface conventions supported by the target AI assistants.

14. Requirements Traceability Matrix

The matrix relates requirements to the architectural elements that satisfy them and to the means by which satisfaction is verified. It is abbreviated here to representative entries; the complete matrix is maintained with the project repository.

Requirement	Satisfied By	Verified By
FR-08 to FR-16	Rule engine (L2); validation orchestration (L3)	Unit tests per rule; integration tests of complete paths
FR-17, NFR-13	Size verification within orchestration; centralized limits (L0)	Boundary tests at each documented limit
FR-20 to FR-23	Enforcement policy handling within orchestration (L3)	Tests covering each policy and each response behaviour
FR-25 to FR-27	Declarative integration mechanism (L5)	Tests of synchronous and asynchronous invocation
FR-31, FR-32	Event recorder (L4); ordering guarantee in orchestration (L3)	Test asserting emission precedes enforcement
NFR-01, NFR-03, NFR-04	Bounded execution runner (L4)	Timed measurement; saturation and delay injection
NFR-07, NFR-08	Resilience controller; local fallback (L4)	Initialization against an unreachable service
NFR-12	Linear-time pattern evaluation engine	Adversarial pattern and input testing
NFR-16	Per-tenant infrastructure within the composition root (L5)	Cross-tenant access testing
NFR-19 to NFR-21	Layer structure and dependency rule	Automated dependency analysis; static inspection

15. Planned Technical Direction

The following capabilities are contemplated by the architecture and are stated here for completeness. They are not implemented at the time of writing and are not within the scope of the requirements specified above. Their inclusion illustrates that the architecture accommodates planned growth without structural revision.

Capability	Architectural Placement	Structural Impact
Service interface for AI assistants	A new entry point at L5	Additive; consumes existing orchestration without modification
Command-line interface	A new entry point at L5	Additive; enables pipeline and version-control integration
Durable event storage	A new adapter at L4 satisfying the existing recorder port	Additive; selected by configuration; core unaffected
Historical query capability	A new orchestration component at L3	Additive; consumes the durable event store
Corrective guidance in results	An extension to the value structures at L2	Additive attribute; deterministic and table-driven
Structural relationship analysis	A new domain component at L2	Additive; extends the categories of expressible requirement
Assistant-specific translation layer	A new port at L1 with adapters at L4	Additive; core remains unaware of assistant identity

Architectural Validation

Every planned capability listed above attaches at the periphery of the system — as a new entry point, a new adapter, or an additive attribute on an existing structure. None requires modification of the validation core or of the orchestration logic. This is the property the architecture was selected to provide, and the fact that the planned roadmap can be expressed entirely as peripheral additions constitutes evidence that the layer boundaries were drawn correctly.

End of Document — CGN-DOC-02, Technical Specification, Version 1.0